PPL Week-8

Rachit Takate – 230962294 – B54

I. Write a program in CUDA to add two Matrices for the following specifications:

a. Each row of resultant matrix to be computed by one thread.

b. Each column of resultant matrix to be computed by one thread.

c. Each element of resultant matrix to be computed by one thread.

```
#include <cuda.h>
#include <stdio.h>

#define M 3
#define N 4

__global__ void addRow(float *aMat, float *bMat, float *resMat, uint m, uint n)
{
    uint i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < m)
        i *= n;
        for(uint j = 0; j < n; j++)
            resMat[i + j] = aMat[i + j] + bMat[i + j];
}

__global__ void addCol(float *aMat, float *bMat, float *resMat, uint m, uint n)
{
    uint j = blockDim.x * blockIdx.x + threadIdx.x;

    if(j < n)
        for(uint i = 0; i < (m - 1) * n; i += n)
            resMat[i + j] = aMat[i + j] + bMat[i + j];
}

__global__ void addEle(float *aMat, float *bMat, float *resMat, uint m, uint n)
{
    uint i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < n * m)
        resMat[i] = aMat[i] + bMat[i];
}

void printMatf(float *mat, uint m, uint n) {
    for(uint i = 0; i < m; i++) {
        for(uint j = 0; j < n; j++)
            printf("%.0f ", mat[i * n + j]);
        printf("\n");
    }
}

int main() {
    float matA[][N] = {
        {0, 1, 2, 3},
        {4, 5, 6, 7},
        {8, 9, 10, 11}
    },
    matB[][N] = {
        {2, 4, 6, 8},
        {10, 12, 14, 16},
```

```
        {18, 20, 22, 24}
    },
    matRes[M][N];
    float *dA, *dB, *dRes;
    size_t size = sizeof(float) * M * N;

    cudaMalloc((void **) &dA, size);
    cudaMalloc((void **) &dB, size);
    cudaMalloc((void **) &dRes, size);

    cudaMemcpy(dA, matA, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, matB, size, cudaMemcpyHostToDevice);

    addRow<<<1, M>>>(dA, dB, dRes, M, N);
    cudaMemcpy(matRes, dRes, size, cudaMemcpyDeviceToHost);
    printf("Row-wise:\n");
    printMatf((float *) matRes, M, N);

    addCol<<<1, N>>>(dA, dB, dRes, M, N);
    cudaMemcpy(matRes, dRes, size, cudaMemcpyDeviceToHost);
    printf("Col-wise:\n");
    printMatf((float *) matRes, M, N);

    addEle<<<1, M * N>>>(dA, dB, dRes, M, N);
    cudaMemcpy(matRes, dRes, size, cudaMemcpyDeviceToHost);
    printf("Ele-wise:\n");
    printMatf((float *) matRes, M, N);

    cudaFree(matA);
    cudaFree(matB);
    cudaFree(matRes);
    printf("Rachit 54\n");
    return 0;
}
```

Output

```
(base) mca@computinglab26-26:~/230962294/PP Lab/Week8$ nvcc Q1.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week8$ ./a.out
Row-wise:
2 5 8 11
14 17 20 23
26 29 32 35
Col-wise:
2 5 8 11
14 17 20 23
26 29 32 35
Ele-wise:
2 5 8 11
14 17 20 23
26 29 32 35
Rachit 54
```

II. Write a program in CUDA to multiply two Matrices for the following specifications:

a. Each row of resultant matrix to be computed by one thread.

b. Each column of resultant matrix to be computed by one thread.

c. Each element of resultant matrix to be computed by one thread.

```
#include <cuda.h>
#include <stdio.h>

#define M 3
```

```
#define N 3
#define P 4

__global__ void mulRow(float *aMat, float *bMat, float *resMat, uint m, uint n,
uint p) {
    uint i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < m)
        for(uint j = 0; j < n; j++) {
            float acc = 0.0f;
            for(uint k = 0; k < p; k++)
                acc += aMat[i * p + k] * bMat[k * n + j];

            resMat[i * n + j] = acc;
        }
}

__global__ void mulCol(float *aMat, float *bMat, float *resMat, uint m, uint n,
uint p) {
    uint j = blockDim.x * blockIdx.x + threadIdx.x;

    if(j < n)
        for(uint i = 0; i < m; i++) {
            float acc = 0.0f;
            for(uint k = 0; k < p; k++)
                acc += aMat[i * p + k] * bMat[k * n + j];

            resMat[i * n + j] = acc;
        }
}

__global__ void mulEle(float *aMat, float *bMat, float *resMat, uint m, uint n,
uint p) {
    uint i = blockDim.x * blockIdx.x + threadIdx.x,
    j = blockDim.y * blockIdx.y + threadIdx.y;

    if(i * n + j < n * m) {
        float acc = 0.0f;
        for(uint k = 0; k < p; k++)
            acc += aMat[i * p + k] * bMat[k * n + j];

        resMat[i * n + j] = acc;
    }
}

void printMatf(float *mat, uint m, uint n) {
    for(uint i = 0; i < m; i++) {
        for(uint j = 0; j < n; j++)
            printf("%.0f ", mat[i * n + j]);
        printf("\n");
    }
}

int main() {
    float matA[][P] = {
        {0, 1, 2, 3},
        {4, 5, 6, 7},
        {8, 9, 10, 11}
    },
    matB[][N] = {
        {2, 10, 18},
        {4, 12, 20},
        {6, 14, 22},
        {8, 16, 24},
```

```
    },
    matRes[M][N];
    float *dA, *dB, *dRes;

    size_t sA = sizeof(float),
    sB = sA * P * N,
    sRes = sA * M * N;
    sA *= M * P;

    cudaMalloc((void **) &dA, sA);
    cudaMalloc((void **) &dB, sB);
    cudaMalloc((void **) &dRes, sRes);

    cudaMemcpy(dA, matA, sA, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, matB, sB, cudaMemcpyHostToDevice);

    mulRow<<<1, M>>>(dA, dB, dRes, M, N, P);
    cudaMemcpy(matRes, dRes, sRes, cudaMemcpyDeviceToHost);
    printf("Row-wise:\n");
    printMatf((float *) matRes, M, N);

    mulCol<<<1, N>>>(dA, dB, dRes, M, N, P);
    cudaMemcpy(matRes, dRes, sRes, cudaMemcpyDeviceToHost);
    printf("Col-wise:\n");
    printMatf((float *) matRes, M, N);

    mulEle<<<1, dim3(M, N)>>>(dA, dB, dRes, M, N, P);
    cudaMemcpy(matRes, dRes, sRes, cudaMemcpyDeviceToHost);
    printf("Ele-wise:\n");
    printMatf((float *) matRes, M, N);

    cudaFree(matA);
    cudaFree(matB);
    cudaFree(matRes);
    printf("Rachit 54\n");
    return 0;
}
```

Output

```
(base) mca@computinglab26-26:~/230962294/PP Lab/Week8$ nvcc Q2.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week8$ ./a.out
Row-wise:
40 88 136
120 296 472
200 504 808
Col-wise:
40 88 136
120 296 472
200 504 808
Ele-wise:
40 88 136
120 296 472
200 504 808
Rachit 54
```